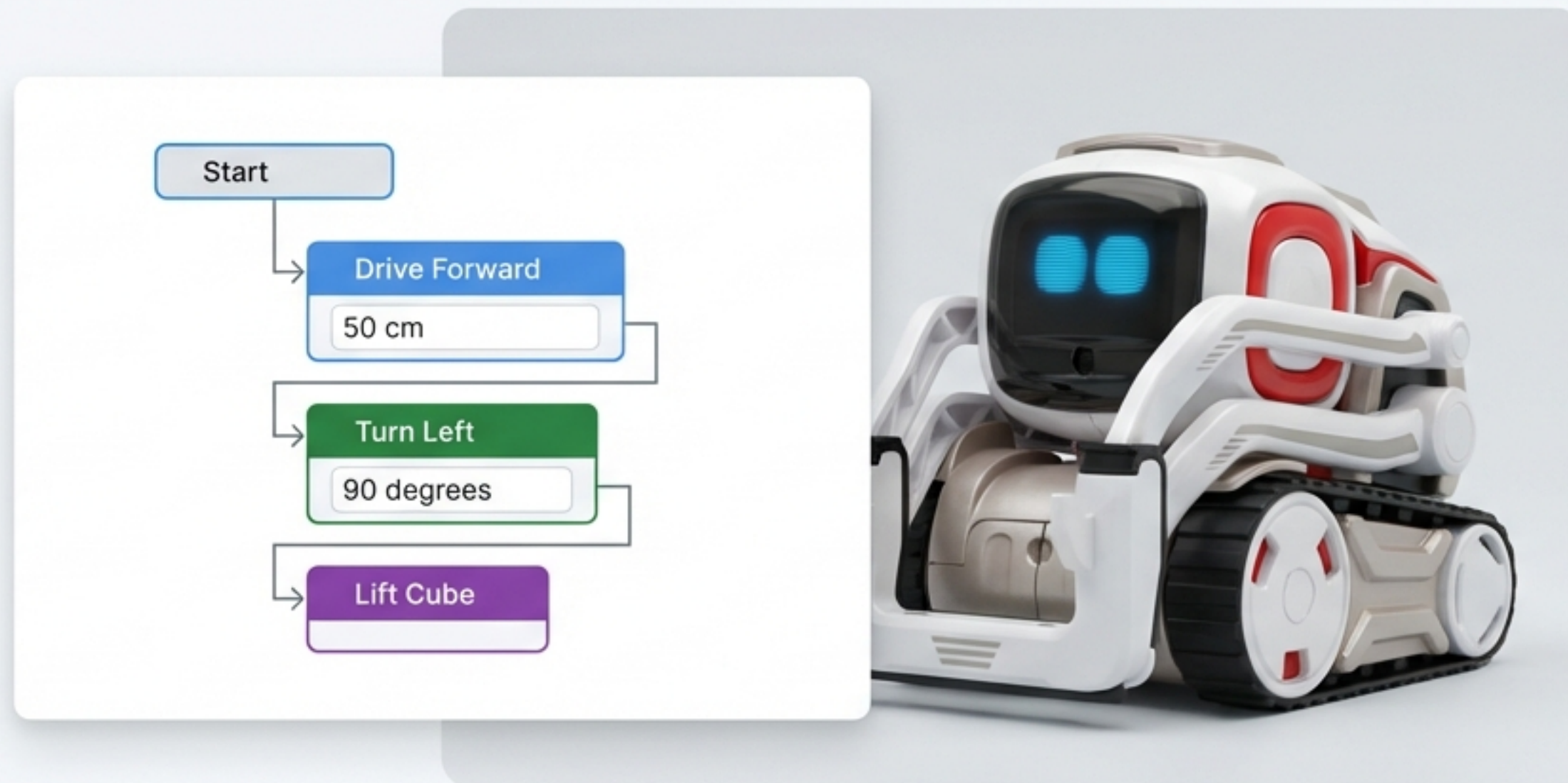
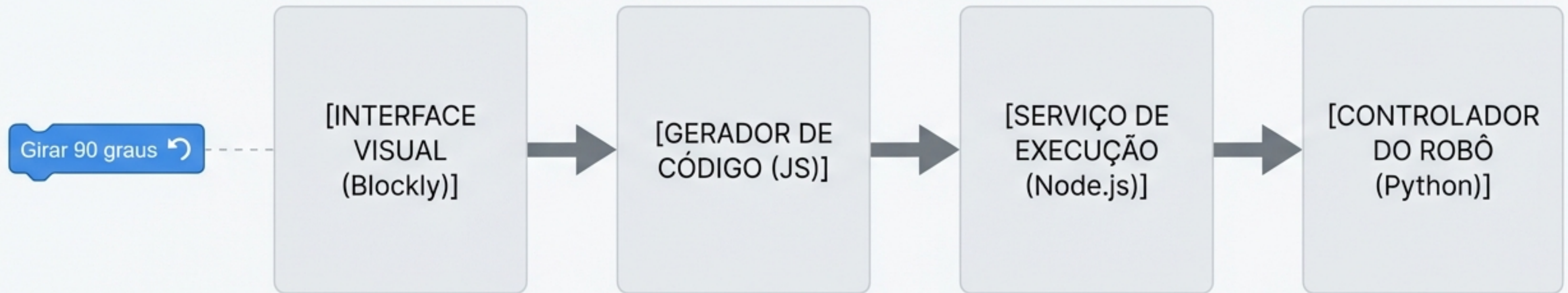


ZoyBlocks: A Anatomia de um Comando

Da Tela ao Movimento: Como um simples bloco visual comanda um robô no mundo real.

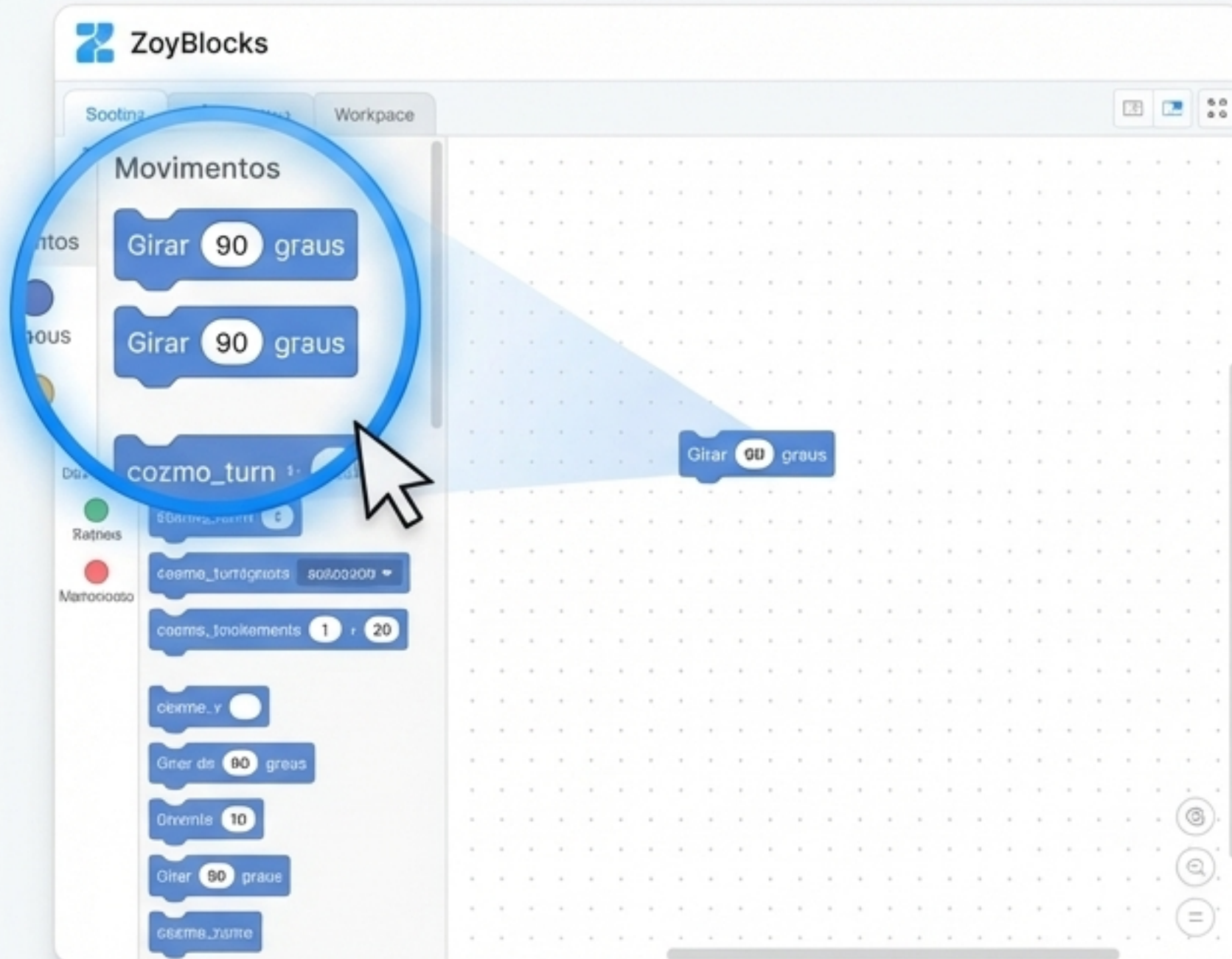


A Arquitetura da Comunicação: Uma Jornada em Quatro Estágios



A jornada de um comando passa por quatro estágios distintos e tecnologias diferentes. Vamos seguir o bloco "Girar" nesta viagem, da sua criação visual até a execução física.

Estágio 1: O Nascimento do Comando na Interface Visual



Tudo começa aqui. O usuário não escreve código, ele combina blocos. O arquivo `home.html` define a aparência e a existência desses blocos através de uma estrutura XML simples e declarativa.

```
<!-- Arquivo: home.html -->
<xml id="toolbox" style="display: none">
  <category name="Movimentos" colour="#5b67a5">
    <block type="cozmo_turn"></block>
  </category>
</xml>
```


Estágio 2: ganhando uma Voz — O Bloco se Transforma em JavaScript



Cada bloco visual possui um 'gerador' correspondente em JavaScript. O arquivo `cozmo_motion.js` é responsável por traduzir a representação visual do bloco em uma linha de código executável que o nosso sistema entenderá.

Girar ↺ Girar 90 graus



```
// Arquivo: cozmo_motion.js

// 1. A definição visual do bloco
Blockly.Blocks['cozmo_turn'] = { /* ... init function ... */ };

// 2. A tradução para código executável
cozmoGenerator.forBlock['cozmo_turn'] = function(block) {
  const angle = block.getFieldValue('ANGLE');
  // Esta função 'virar' será o elo com o próximo estágio.
  return `await virar(${angle});`;
};
```

Este é o artefato que segue para o próximo estágio.

A Ponte Entre Dois Mundos: Do Navegador ao Hardware



O código JavaScript gerado no navegador (frontend) não pode controlar hardware diretamente por razões de segurança e limitações da plataforma. Precisamos de um intermediário seguro e poderoso no servidor (backend) para receber essa instrução e traduzi-la para o robô: um **serviço Node.js**.

Estágio 3: A Sala de Controle — O Serviço Node.js Interpreta o Comando



O `blockly\_service.js` atua como o cérebro. Ele recebe o código gerado, o executa em uma VM segura para isolar o processo, e mapeia a função `virar()` para um comando JSON específico, pronto para ser enviado ao controlador do robô.

```
// Arquivo: blockly_service.js
const cozmoActions = {
  virar: async (angulo) => {
    // O texto 'virar' é mapeado para um objeto estruturado
    const comando = { cmd: "turn", angle: parseInt(angulo) };
    enviarParaPython(comando); // O comando é empacotado e enviado
    // Espera o robô terminar o movimento físico
    return new Promise(resolve => setTimeout(resolve, 1500));
  }
};
```

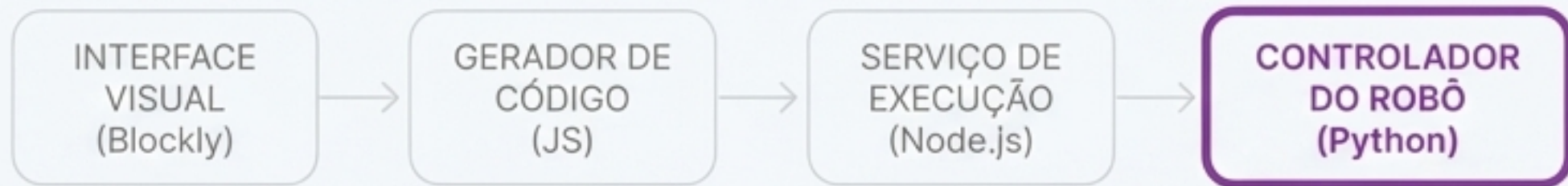
```
// O código do usuário é executado em um contexto seguro e controlado
const contexto = vm.createContext({ ...cozmoActions });
await script.runInContext(contexto);
```

A função do navegador é mapeada para uma ação segura no servidor.

A Linha Direta: Como Node.js e Python Conversam



A comunicação entre o serviço e o script do robô é feita da forma mais direta e eficaz possível. O Node.js escreve o comando JSON em sua saída padrão (`stdout`), e o script Python o lê continuamente de sua entrada padrão (`stdin`). É um canal de comunicação universal e robusto entre processos.



Estágio 4: Mãos à Obra — O Python Comanda os Motores

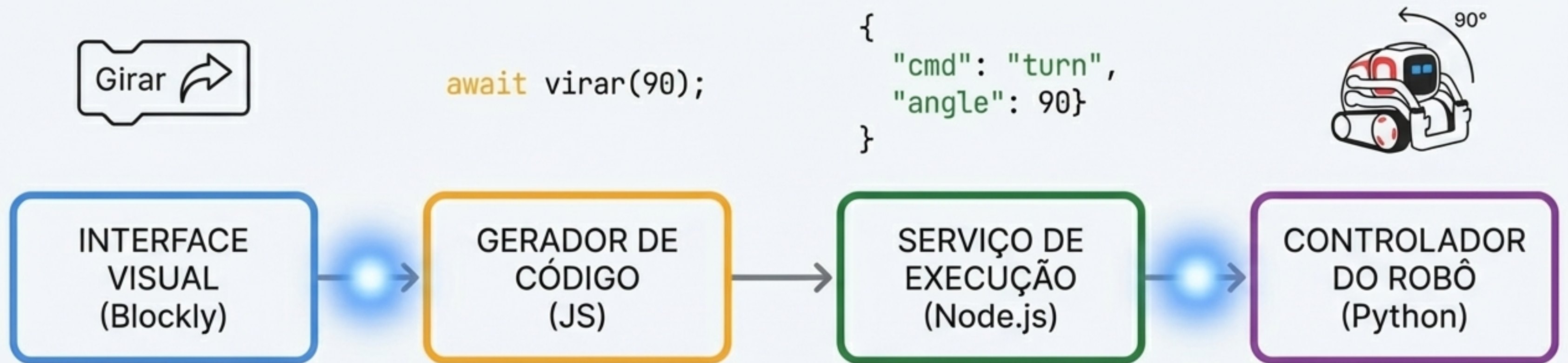
O script `cozmo_server.py` escuta por comandos JSON em um loop infinito. Ao receber `{'cmd': 'turn'}`, ele calcula a duração e a direção necessárias e traduz isso em comandos de baixo nível para os motores das rodas usando a biblioteca PyCozmo.

```
# Arquivo: cozmo_server.py
elif cmd == "turn":
    angle = data.get("angle", 0)
    turn_speed = 30
    # A "fórmula secreta": como um ângulo vira tempo
    duration = abs(angle) / 85.0
    if angle > 0: # Girar para a esquerda
        cli.drive_wheels(-turn_speed, turn_speed, duration=duration)
    else: # Girar para a direita
        cli.drive_wheels(turn_speed, -turn_speed, duration=duration)
```

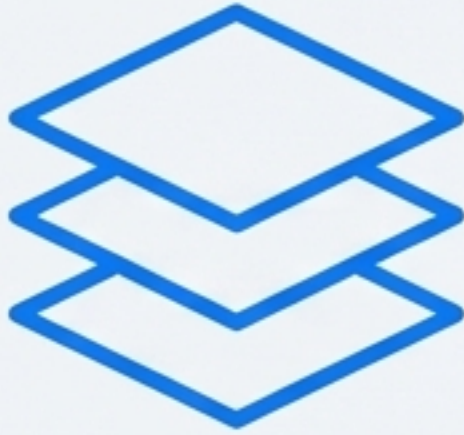
A "fórmula secreta" que transforma um conceito (ângulo) em uma realidade física (tempo).



A Jornada Completa: Da Tela ao Movimento



Checklist de Arquitetura: Princípios-chave para Replicar



Separação Clara de Responsabilidades

A interface (UI) não conhece o hardware. A lógica de negócios (Backend) não conhece a UI. O controlador (Python) só conhece comandos diretos.



Tradução Progressiva de Intenção

O comando evolui de uma intenção visual (Bloco) para um comando de alto nível (JS), para um pacote de dados (JSON), e finalmente para uma ação de baixo nível (motores).



Comunicação Robusta e Simples

Use formatos universais (JSON) e canais padrão (stdio) para garantir a interoperabilidade entre diferentes linguagens e processos.



Execução Segura em Ambientes Isolados

Nunca confie no código gerado pelo usuário. Execute-o em um ambiente controlado, como uma VM, expondo apenas as funções seguras necessárias.

ZoyBlocks



ZoyBlocks: Da Tela ao Movimento

Explore o projeto em github.com/exemplo/zoyblocks